

Technical Design Document (TDD)

BMI4242 — Game Programming / Term Project

Document Header

Field	Information
Game Title	Cubestone — Voxel-Based Action RPG
Team Name	Cubestone
Target Engine / Framework	Python 3.12 + Ursina 8.3.0 (Panda3D rendering backend)

Team Members, IDs and Primary Roles (roles are mapped to the actual modules in the codebase)

Member	Student ID	Primary Role	Owned code
Arif Can Ekici	2111504014	Core Engine, Physics & Persistence	core/world , core/chunk , main (game loop, chunk streaming, custom physics), persistence/save_manager
Hazan Ezgi Erçin	2311504299	Procedural Generation & Rendering	core/generation , core/dungeon , core/structures , render/mesh , render/chunk_renderer
Özge Yeşilsancak	2211504044	Gameplay Systems, UI & Audio	systems/ (combat, AI, loot, crafting, abilities, equipment), entities/ , ui/ , audio (ui/sfx , tools/gen_sounds)

Note: the codebase is ~7,000 lines of Python across 40+ modules. Although the roles overlap, each member owns at least one subsystem; all members contributed to shared integration, debugging and tests.

1. Game Overview

Elevator Pitch: A first-person voxel action-RPG set in a procedurally generated, fully destructible/buildable world, where the player picks a class (Knight / Ranger / Mage), hunts enemies, explores underground dungeons and sky citadels, and fights bosses.

Core Mechanics: - **Block interaction:** break and place blocks with the mouse (raycast-based). - **First-person movement:** WASD + jump/double-jump, sprint (Shift) and swimming — all driven by **custom physics** (the engine's built-in controller is disabled). - **Combat:** melee attack (**F**) plus two class-specific active abilities (**Q** / **R**) on cooldowns. - **Progression:** XP from kills → level-up → increased health; loot collection, crafting, equipment and a merchant shop. - **Exploration:** a 5-floor underground dungeon, above-ground structures, boss arenas linked by teleport portals, and a day/night cycle.

Win / Loss Conditions: - **Loss:** when the player's health reaches 0 a death screen appears and, after ~2.5 s, the player **respawns** at the spawn point (no permanent death — a sandbox progression loop). - **Goal / Win:** defeating the increasingly difficult bosses found deeper in the dungeon and clearing structure arenas. Progression is designed as an endless loop; instead of a hard "You Win" screen, every boss defeated is an intermediate objective.

2. Software Architecture

2.1 High-Level Structure — Layered Architecture + "Data / Visual Separation"

The guiding architectural principle is summarised in one sentence: **separate data from presentation**. The world is a pure data structure in memory; what is drawn on screen is one *visualisation* of that data at a given

moment. The `core/` layer **does not know about the game engine** (Ursina) at all and can run — and be tested — without it.

Instead of a deep OOP inheritance hierarchy, the project uses a **layered + lightweight component-style** approach. Dependencies flow in one direction only — inward, toward `core` — with no circular dependencies.

```
src/
├─ main.py      → composition root: wires everything together, holds no game logic
├─ core/        → ENGINE-INDEPENDENT pure logic (world, chunk, generation, dungeon, structures)
├─ render/      → turns data into a mesh and draws it (mesher, chunk_renderer)
├─ entities/    → player state, enemies, bosses, merchant, chests
├─ systems/     → combat, inventory, loot, crafting, abilities, spawning, equipment, day/night
├─ ui/          → HUD, hotbar, minimap, inventory/crafting/shop screens, sound service
├─ persistence/ → JSON save/load
config/         → NOT code: block, class and settings definitions (data-driven)
tests/          → unit tests for the pure core logic (pytest)
```

Layer	One-sentence responsibility	Does not know about
core	Voxel data, chunks, procedural generation	The screen, Ursina
render	Convert data to a mesh and draw it, face culling	Game rules
entities	Player/enemy state, movement, AI	Rendering details
systems	Combat, inventory, loot, ability rules	The visuals
ui	HUD, menus	Internal game logic
persistence	Write/read world & player to disk	The internals of other layers

2.2 Core Managers / Controllers

The single entry point is `main.py`; it performs no work itself, only constructs the systems and wires them together. At runtime the game loop runs inside **Ursina's** `app.run()` and every system is an `Entity` whose `update()` / `input()` method is invoked each frame (the **Update Method** pattern). The principal runtime systems are:

- **World** — the voxel data store (`get_block` / `set_block`, world↔chunk coordinate conversion).
- **TerrainGenerator / DungeonGenerator / StructureGenerator** — fill chunks, place dungeons and structures.
- **ChunkStreamer** — as the player walks, queues new columns and generates/renders them on a **per-frame budget**; destroys renderers that fall out of range.
- **MovementSystem / TerrainPhysics** — horizontal wall collision and vertical physics (custom-written; the engine's controller is disabled via `speed=0`).
- **SpawnManager / CombatHandler / AbilitySystem** — enemy spawning, melee combat, active abilities.
- **BlockInteraction** — raycast to break/place blocks and notify persistence of the change.
- **HUD / Hotbar / Minimap / *UI** — the interface; it listens to game state via **callbacks**.

Inter-module communication — loose coupling: systems do not call each other directly; they use **callbacks/events** instead. For example, when an enemy dies an `on_death(enemy, type)` callback fires; the `SpawnManager` catches it to drop loot and grant XP — the `Enemy` class never knows the loot system exists. Likewise, changing equipment triggers `on_change` → the HUD updates, and opening a chest calls `on_collect`. This is the **Observer** pattern.

2.3 Design Patterns Used

Pattern	Where	Why
Update Method / Game Loop	Every system's <code>Entity.update()</code>	Advance state each frame; hook into the engine loop from one place
Observer (callback)	<code>on_death</code> , <code>on_change</code> , <code>on_collect</code> , <code>on_select</code>	Let systems communicate without tight coupling
State Machine	Death/respawn, day/night cycle, ability cooldowns	Cleanly manage time-based transitions (alive→dead→respawn)
Data-Driven / Strategy	Classes as <code>dataclass</code> + <code>config/*.json</code>	Decouple content from code; add a block/class without changing code
Composition Root	<code>main.py</code>	Construct and inject all dependencies in one place (dependency injection)
Component	Enemy body/eyes/HP-bar as separate <code>Entity</code> objects	Build visuals from independent components

3. Algorithmic Implementation & Data Structures

This is the most engineering-intensive section. By design, no off-the-shelf engine "level editor" was used; the **world, collision, generation and meshing are all hand-written**.

3.1 Data Structures

- **World = dictionary (hash map):** `World._chunks` , keyed `(cx, cy, cz) → ChunkData` . Stores an unbounded world *sparsely* — only generated chunks live in memory. World↔chunk conversion is `divmod(w, chunk_size)` , i.e. $O(1)$.
- **Chunk = NumPy 3-D array:** each `ChunkData` is a `16×16×16 uint16 numpy` array. 2 bytes per block, ~8 KB per chunk; the contiguous buffer makes access fast and `is_empty()` a single `np.any` call.
- **Dungeon air set:** dungeon cavities are **precomputed once** into a `set[(wx,wy,wz)]` . At runtime, "is this block dungeon air?" is **$O(1)$** (noise is not re-run on every chunk).
- **Mesh intermediate:** `MeshData` — parallel lists of `vertices / triangles / uvs / normals / colors` , fed directly into an Ursina `Mesh` .
- **Corridor plan = graph:** dungeon rooms are nodes and corridors are edges; a connected graph is built via nearest-neighbour linking.
- **Inventory/state:** `block_counts` / `item_counts` dictionaries; enemies/bosses are lists inside `SpawnManager` .

3.2 Procedural Generation

Terrain height — multi-octave OpenSimplex noise: `TerrainGenerator.get_surface_height` sums three octaves of simplex noise with `1.0 / 0.5 / 0.25` weights (fractal-Brownian-motion style). A separate low-frequency "biome" noise selects one of four biomes (Plains/Desert/Snow/Forest). To avoid cliffs at biome borders, biome weights are blended over a transition band using **smoothstep** `(t·t·(3-2t))` ; the final height is the weighted average of the four biome heights.

Water: depressions below sea level (`y=33`) are flooded with water blocks during generation.

Caves — "worm" tubes: for two independent 3-D noise fields `n1, n2` , voxels satisfying `n12 + n22 < threshold` are carved out. The intersection of the two fields produces **connected tunnel networks** rather than isolated pockets.

Ores and trees — integer hash instead of noise: rather than calling expensive noise per ore/tree, a cheap integer hash is used: `(wx·374761393 ^ wy·1376312589 ^ wz·2654435761 ^ seed) & 0xFFFF` . Comparing the

result to a threshold yields a deterministic yet "random-looking" distribution (coal ~3%, iron ~1.5%; trees ~2% in forest). This noticeably speeds up generation.

Dungeon — graph-based room/corridor generation: `DungeonGenerator` places rooms with some probability on a grid across 5 floors (`FLOOR_Y = [22,14,6,-4,-14]`); deeper floors are denser and tighter). Then: - **Horizontal connection:** each room is linked to its **nearest neighbour on the same floor** with an L-shaped (3×3 cross-section) corridor → a connected graph. - **Vertical connection:** adjacent floors are linked by ramp tunnels whose slope **never exceeds 1:2** (so the player can walk between levels without jumping). The step count is chosen as `max(|dx|, |dz|, |dy|·2)` to guarantee the gradient. All cavities are carved once into the "air set"; terrain generation then consults that set and clears the corresponding blocks.

3.3 Rendering: Mesh Generation via Face Culling

A naive approach draws every cube with 12 triangles and wastes the GPU on hidden interior faces. Instead, `mesher.build_mesh` emits **only faces whose neighbour is air (0)**. When the neighbour is inside the same chunk it is read from the array; when it is on a chunk boundary it is fetched **across chunks** via `neighbor_getter` (the world's `get_block`) — so no holes/seams appear on adjacent chunk surfaces. **Vertex colors** are used instead of textures (block type → color), and a fixed per-face brightness factor (`_FACE_DIM`) is applied to give a cheap fake lighting/ambient-occlusion effect. Each chunk is drawn as a **single Entity /mesh** (batched, rather than thousands of cubes).

3.4 Custom Physics and Collision — "path scanning" (anti-tunneling)

A custom system that queries voxel data directly is used instead of the engine's physics: - **Vertical physics (TerrainPhysics):** the classic "is there a block at the new y?" check tunnels through the ground at high speed. Instead, the system **scans every block along the fall path** and finds the first solid surface it would hit. A 5-column check around the feet also prevents hanging on edges; buoyancy in water, double jump, **fall damage** above a velocity threshold, and knockback are all handled here. - **Horizontal collision (MovementSystem):** movement is attempted **axis by axis** — X first, then Z. If `world.get_block` returns solid on an axis, that axis is cancelled, so the player slides along walls instead of getting stuck in corners. Swim slowdown and sprint multipliers are applied here.

3.5 Enemy AI and Combat

- **AI (seek/steering):** `Enemy.update` moves toward the player each frame along a normalised vector, attacks on a cooldown once in range, and can be slowed by a "slow" status effect. Direct seeking was chosen over A*: in the open voxel terrain the target is usually in line of sight, and A*'s per-frame path cost is unnecessary at this scale (a deliberate architectural decision — see §4).
- **Depth-based spawning:** `SpawnManager` looks at the player's Y height and picks an appropriate enemy: slimes/goblins on the surface, trolls in the shallow dungeon, skeletons in the deep. At most `MAX_ENEMIES = 6` enemies exist at once.
- **Melee:** `CombatHandler` damages all targets within a reach sphere (`REACH = 3.0`) (base damage + equipment bonus) and shows a damage-number particle.

3.6 Persistence — Delta Save

`SaveManager` does not write the whole world to disk — that would be enormous. Instead it stores only the **blocks the player has modified** (`"x,y,z": id`) as JSON; the rest of the world is regenerated **deterministically** from the seed. Player state (class, consumables, equipment, gear bag) is written separately in a `version: 2` format. Loading first regenerates the world, then applies the delta blocks on top.

4. Optimization & Memory Management

Performance bottlenecks and solutions:

Bottleneck	Solution
Hidden cube faces	Face culling: only faces adjacent to air are added to the mesh (interior faces are never generated)
Thousands of cubes = thousands of draws	One mesh entity per chunk; vertex colors instead of textures → no texture binds
Frame freeze while generating new area	Budgeted streaming: <code>ChunkStreamer</code> generates at most <code>GEN_PER_FRAME</code> columns per frame; the rest wait in a queue → no FPS spikes
FPC raycast traversing the whole scene tree	<code>player.traverse_target</code> redirected to an empty entity → raycasts return instantly
A mesh collider on every chunk	Colliders are off on most chunks (<code>with_collider=False</code>); collision is already resolved via <code>world.get_block</code>
Repeated noise per chunk	Dungeon cavities are a precomputed set (O(1) query); ores/trees use integer hashing (no noise)
Voxel storage	NumPy <code>uint16</code> contiguous array — compact and fast

Memory management: memory usage is **bounded by the render distance**. As the player moves, `ChunkStreamer` recomputes the target set and destroys chunk renderers outside the range via `destroy()`; thus the number of loaded meshes stays constant no matter how far the map is explored. When enemies die, loot is collected, or chests are opened, the corresponding `Entity` objects are destroyed. The enemy count is capped at `MAX_ENEMIES = 6`, and concurrent boss/loot counts are bounded by the structure/spawn logic. (Explicit **object pooling** for projectiles/enemies was not implemented; the concurrent object count is kept low and spawning is cooldown-limited — a deliberate simplification that keeps GC pressure low in practice, and a candidate for future optimization.)

5. Version Control & Workflow

Git strategy — Feature Branch Workflow: `main` is kept in a working state at all times. Each new system (e.g. "dungeon generation", "ability system", "minimap") is developed on its own branch and merged into `main` once it works; half-finished work is not pushed to `main`. The layered architecture makes this division of labour natural: two people can work on `render/` and `systems/` simultaneously without conflicts because the file boundaries are clear. Every major milestone is marked with a version `tag`; if something breaks, the team reverts to the last working version.

Generated files kept out of the repo: the `.gitignore` excludes the virtual environment (`venv/`), Python cache (`__pycache__`), game saves (`saves/`), logs and IDE/OS junk — these would bloat the repo and produce needless conflicts.

Integration challenges and how they were resolved:

- **Chunk-boundary seams:** while two people worked in parallel on `core` (data) and `render` (mesh), incorrectly culled faces at chunk borders produced visible "seams/holes". The fix was to inject a `neighbor_getter` into the mesher so it consults the neighbouring chunk's data — which clarified the contract between the data and render layers.
- **Physics ↔ streaming race:** a player falling onto a not-yet-generated column would sink into the void. The fix was to make `TerrainPhysics` operate only on generated columns (`_gen_cols`) and zero the vertical velocity over an ungenerated column.
- **Merge discipline:** thanks to the "data/visual separation" principle, conflicts mostly stayed within a single layer; because `core/` is pure logic, its unit tests (`pytest` , 14 tests) caught regressions before every merge.

Appendix: Testability

The concrete reward of keeping `core/` engine-independent: without opening a window, `pytest` can verify chunk data management, world coordinate conversion and procedural generation (`tests/` — 14 tests). No tests were written for the render and input layers; the real payoff is verifying the correctness of the **rules**.

This document explains the "under-the-hood" engineering of the game, not its story or atmosphere. There is a single architectural principle: separate data from presentation — that is what makes half the system testable and the other half replaceable.